CognEn v2.0 — Multi-Person CCTV Room Analyzer

> ***Server-based, multi-person cognitive engagement analysis for classrooms*** *For the single-user browser version, see* ***v1.0 README****.*

Table of Contents

1. Overview
2. Architecture
3. Perception Layer
4. Tracking Layer
5. Feature Extraction
6. Behavior Classifier (1D-CNN)
7. Training Pipeline
8. Context-Gate Override
9. Room Analytics
10. Video Input Sources
11. FastAPI Server & WebSocket
12. Tech Stack
13. Repository Structure
14. Setup & Installation
15. CLI Usage
16. REST API Reference
17. Testing & Evaluation
18. Performance Metrics
19. Datasets
20. Privacy
21. Cross-Reference: v1.0

---

1. Overview

CognEn v2.0 scales the v1.0 single-user pipeline to **multi-person CCTV room analysis**. It processes RTSP streams, video files, or USB cameras at 1280×720 @ 15 FPS using a Python/FastAPI backend with GPU acceleration.

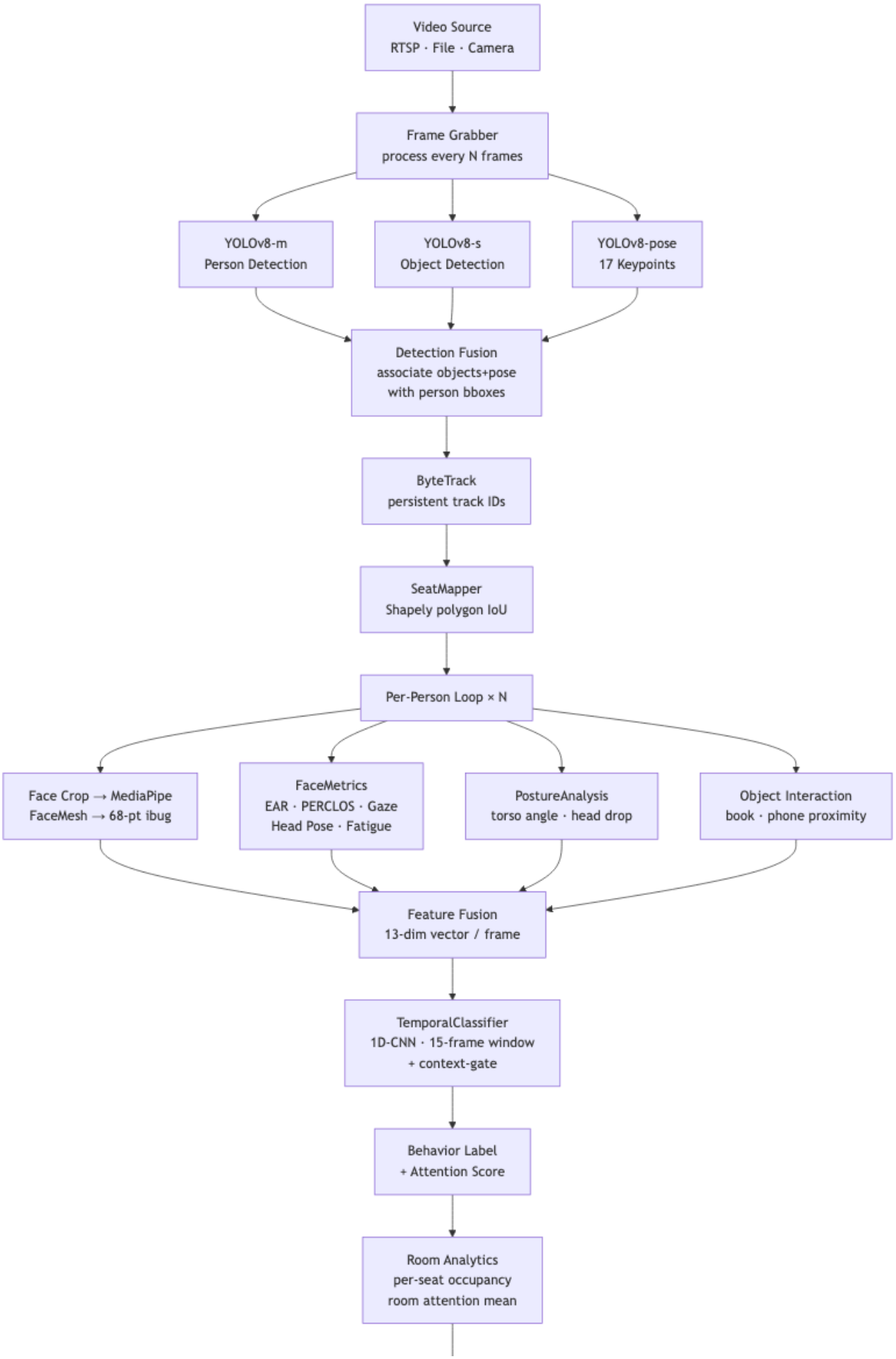
Core equation: `v2.0 = multi_person_detection + tracking + [v1.0_per_person_pipeline × N] + room_analytics`

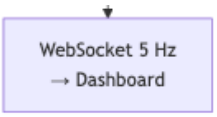
Each tracked person gets their own independent instance of the v1.0 ML pipeline (BlinkDetector, HeadPoseTracker, FatigueCalculator, BehaviorClassifier).

---

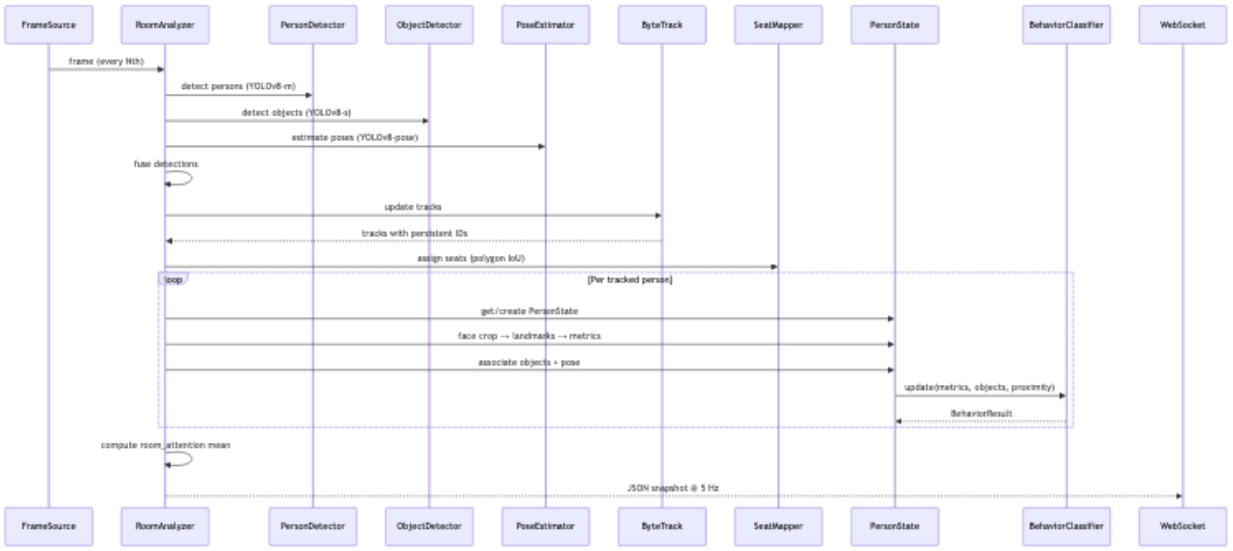
2. Architecture

2.1 Full Pipeline


Video Source
RTSP · File · Camera
Frame Grabber
process every N frames
YOLOv8-m
Person Detection
YOLOv8-s
Object Detection
YOLOv8-pose
17 Keypoints
Detection Fusion
associate objects+pose
with person bboxes
ByteTrack
persistent track IDs
SeatMapper
Shapely polygon IoU
Per-Person Loop × N
Face Crop → MediaPipe
FaceMesh → 68-pt ibug
FaceMetrics
EAR · PERCLOS · Gaze
Head Pose · Fatigue
PostureAnalysis
torso angle · head drop
Object Interaction
book · phone proximity
Feature Fusion
13-dim vector / frame
TemporalClassifier
1D-CNN · 15-frame window
+ context-gate
Behavior Label
+ Attention Score
Room Analytics
per-seat occupancy
room attention mean



2.2 Sequence Diagram



---

3. Perception Layer

3.1 Person Detection — YOLOv8-m

Property	Value
Model	`yolov8m.pt` (25.9M params)
Why chosen	Best mAP/speed for medium scenes; handles occlusion well
Confidence	≥ 0.40
Inference	~8ms/frame GPU
Alternatives	RT-DETR (heavier), Faster R-CNN (slower)

3.2 Object Detection — YOLOv8-s

Property	Value
Model	`yolov8s.pt` (11.2M params)
Target classes	book, phone, laptop, pen, notebook
Confidence	≥ 0.25
Inference	~4ms/frame GPU

3.3 Pose Estimation — YOLOv8-pose

Property	Value

Model	`yolov8m-pose.pt`
Keypoints	17 COCO (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles)
Confidence	≥ 0.40
Inference	~6ms/frame GPU

Posture Analysis from pose keypoints:

```
torso_vector = shoulder_midpoint − hip_midpoint
lean_angle = angle_between(torso_vector, vertical)
is_upright = lean_angle < 15°
is_head_dropped = nose.y < shoulder_mid.y
```

3.4 Face Landmarks — MediaPipe FaceMesh

- 478 facial landmarks detected via MediaPipe Tasks API (VIDEO mode)
- Re-mapped to **68-point ibug convention** for compatibility with v1.0 face_metrics
- Mapping covers: jaw (17), eyebrows (10), nose (9), eyes (12), mouth (20)

---

4. Tracking Layer

4.1 ByteTrack

Property	Value
Library	`boxmot` (includes ByteTrack)
Why chosen	SOTA MOT; handles occlusion via low-confidence detections; no ReID needed
Tracking	~1ms/frame CPU
Target MOTA	≥ 75%
Target IDF1	≥ 80%
Target ID switches	< 5/min

4.2 Seat Mapper

Pre-configured seat zones as **Shapely polygons** in normalized [0,1] coordinates.

Assignment algorithm:

1. Primary: centroid containment (person bbox center inside seat polygon → score 1.0)
2. Fallback: IoU overlap (intersection/union ≥ 0.30)
3. If two persons overlap same seat, highest score wins

Example seat map (`data/seat_maps/room_default.json`):

```
{
  "seat_1": [[0.05, 0.10], [0.30, 0.10], [0.30, 0.90], [0.05, 0.90]],
  "seat_2": [[0.35, 0.10], [0.60, 0.10], [0.60, 0.90], [0.35, 0.90]],
  "seat_3": [[0.65, 0.10], [0.95, 0.10], [0.95, 0.90], [0.65, 0.90]]
}
```

---

5. Feature Extraction

Python port of v1.0 `face-geometry.ts`. Per-person stateful detectors:

Class	State	Function
`BlinkDetector`	EAR history (120s), blink events	Blink detection, blink rate, PERCLOS
`HeadPoseTracker`	Pose history (30s)	Stability score, fidget score, pose variance
`FatigueCalculator`	PERCLOS/blink/EAR history (5min)	Composite fatigue index (0–100)

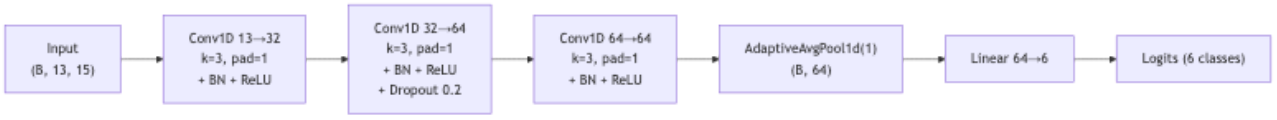
13-dim feature vector (identical to v1.0 for model compatibility):

#	Feature	Normalization
0	gaze_x	[-1, 1]
1	gaze_y	[-1, 1]
2	pitch_norm	pitch/90 → [-1, 1]
3	yaw_norm	yaw/90 → [-1, 1]
4	ear	[0, 1]
5	perclos	/100 → [0, 1]
6	stability	/100 → [0, 1]
7–10	object one-hots	book, phone, laptop, none
11	fidget	/100 → [0, 1]
12	proximity_code	0.0–1.0

---

6. Behavior Classifier (1D-CNN)

6.1 Model Architecture



Property	Value
Parameters	20,550
Input shape	`(B, 13, 15)` channel-first
Output	6-class logits
Receptive field	7 frames = 1.4 seconds
Inference latency	0.02 ms CPU
ONNX opset	17

Weight init	Kaiming (conv), Xavier (linear)

6.2 Six Behavior Classes

Class	Score	Description
`attentive_screen`	85	Watching lecture/screen
`attentive_reading`	80	Reading book/notes
`distracted`	45	Looking around, talking
`using_phone`	25	Phone distraction
`drowsy`	20	Head down + low EAR
`absent`	0	Seat empty

---

7. Training Pipeline

7.1 Training CLI

```
# Realistic synthetic data
python3 v2.0/training/train_classifier.py \
    --realistic \
    --realistic-n-per-class 2000 \
    --realistic-augment 3 \
    --penalty-loss \
    --epochs 100 --batch 256

# With real datasets
python3 v2.0/training/train_classifier.py \
    --daisee /data/daisee \
    --engagenet /data/engagenet \
    --penalty-loss --epochs 100
```

7.2 ConfusionPenaltyLoss

Asymmetric loss penalizing costly confusions:

True → Predicted	Penalty	Rationale
phone → reading	3.0×	Hidden phone use (safety)
reading → phone	2.5×	Student unfairly penalized
absent → screen	2.0×	Absent student missed
All other pairs	1.0×	Standard

7.3 Training Details

Setting	Value
Optimizer	AdamW (lr=1e-3, weight_decay=1e-4)
Scheduler	CosineAnnealingLR (η_min = lr×0.01)

Label smoothing	0.05
Class weighting	Inverse-frequency
Early stopping	Patience 10 epochs
Validation split	15%
Gradient clipping	max_norm = 1.0

7.4 ONNX Export

```
python3 v2.0/training/export_onnx.py \
    --checkpoint v2.0/models/behavior_classifier.pt \
    --output v2.0/models/behavior_classifier_v2.onnx \
    --verify
```

---

8. Context-Gate Override

Post-processes CNN softmax probabilities with hard object-context rules:

Condition	Action
Phone detected + holding_phone	Boost `using_phone` to ≥0.85, suppress `attentive_reading` to ≤0.05
Book detected + holding_book, no phone	Boost `attentive_reading` to ≥0.80, suppress `using_phone` to ≤0.08
Downward gaze + no objects	Compress confidence: `p = p×0.7 + uniform×0.3`

---

9. Room Analytics

Per-frame output structure:

```
{
  "ts": 1741519810.4,
  "persons": [
    {
      "track_id": 1,
      "seat_id": "seat_2",
      "behavior": "attentive_screen",
      "attention_score": 82.5,
      "confidence": 0.94
    }
  ],
  "occupancy": {
    "occupied": ["seat_2", "seat_4"],
    "absent": ["seat_1", "seat_3", "seat_5", "seat_6"],
    "total": 6
  },
  "room_attention": 82.5
}
```

---

10. Video Input Sources

Unified `FrameSource` abstraction supporting three source types:

Source	Class	Key Config
RTSP	`RTSPSource`	URL, transport (tcp/udp), auto-reconnect
File	`VideoFileSource`	Path, loop option, encoded FPS pacing
Camera	`CameraSource`	Device index, resolution, FPS

Frame-skip: `--process-every N` analyses 1 in N frames. At 15 FPS with N=3 → ~5 analysis FPS.

---

11. FastAPI Server & WebSocket

```
uvicorn server.main:app --host 0.0.0.0 --port 8000
```

WebSocket pushes room analytics at **5 Hz** to connected dashboard clients.

---

12. Tech Stack

Layer	Technology
API server	FastAPI + uvicorn
Person detection	YOLOv8-m (Ultralytics)
Object detection	YOLOv8-s
Pose estimation	YOLOv8-pose (17 COCO keypoints)
Face landmarks	MediaPipe FaceMesh → 68-pt ibug
Tracking	ByteTrack (boxmot)
Seat mapping	Shapely polygon IoU
Behavior classifier	1D-CNN ONNX (20,550 params)
Video input	OpenCV VideoCapture
Dashboard	FastAPI WebSocket @ 5 Hz
Training	PyTorch + ONNX export

---

13. Repository Structure

```
v2.0/
├── server/
│   ├── main.py                # FastAPI app, REST + WebSocket
│   ├── config.py              # Settings, model paths, thresholds
│   └── requirements.txt
├── input/
│   └── frame_source.py        # FrameSource ABC: RTSP, File, Camera
├── perception/
│   ├── person_detector.py     # YOLOv8-m
│   ├── object_detector.py     # YOLOv8-s
```

```
│   ├── pose_estimator.py        # YOLOv8-pose + posture analysis
│   └── face_landmarks.py        # MediaPipe FaceMesh → 68-pt ibug
├── tracking/
│   ├── byte_tracker.py          # ByteTrack wrapper (boxmot)
│   └── seat_mapper.py           # Seat zone assignment (Shapely)
├── features/
│   └── face_metrics.py          # Python port of v1.0 face-geometry.ts
├── inference/
│   ├── behavior_classifier.py   # Temporal 1D-CNN + context-gate
│   └── room_analyzer.py         # Main orchestrator
├── training/
│   ├── model.py                 # BehaviorClassifierCNN (20,550 params)
│   ├── train_classifier.py      # Training CLI
│   ├── export_onnx.py           # ONNX export + verification
│   ├── feature_extractor.py     # 13-dim feature vector
│   └── datasets/
│       ├── base_dataset.py
│       ├── realistic_synthetic_loader.py
│       ├── daisee_loader.py
│       ├── engagenet_loader.py
│       └── classroom_loader.py
├── evaluation/
│   ├── evaluate.py              # Full evaluation runner
│   ├── behavior_metrics.py      # Accuracy, F1, temporal consistency
│   ├── tracking_metrics.py      # MOTA, IDF1, ID switches
│   ├── occupancy_metrics.py     # Occupancy accuracy, absence latency
│   ├── latency_benchmark.py     # P95 latency
│   └── reports/
├── models/
│   ├── behavior_classifier_v2.onnx
│   └── download_models.py
├── data/seat_maps/
│   └── room_default.json
├── tests/                       # 61 pytest tests
├── run_analyzer.py              # CLI entry point
├── yolov8m-pose.pt
└── yolov8s.pt
```

---

14. Setup & Installation

```bash
cd v2.0
python3 -m venv .venv && source .venv/bin/activate
pip install -r server/requirements.txt

# GPU (NVIDIA Linux/Windows only):
# pip install onnxruntime-gpu>=1.18.0

# Download YOLO weights
python3 models/download_models.py
```

---

15. CLI Usage

```
# RTSP
python3 run_analyzer.py --source rtsp \
    --url "rtsp://admin:pass@192.168.1.100:554/ch1" \
    --room room_default --process-every 3 --show

# Video file
python3 run_analyzer.py --source file \
    --url /data/classroom.mp4 --room room_default --loop

# Webcam
python3 run_analyzer.py --source camera --device 0 --show

# JSON output
python3 run_analyzer.py --source camera --print-json | jq '.room_attention'
```

---

16. REST API Reference

Method	Path	Description
`POST`	`/rooms/{room_id}/start`	Start analysis (SourceConfig body)
`POST`	`/rooms/{room_id}/stop`	Stop analysis
`GET`	`/rooms/{room_id}/snapshot`	Latest analytics JSON
`GET`	`/rooms`	List all rooms
`WS`	`/ws/room/{room_id}`	Real-time push @ 5 Hz

Environment variables: `COGNEN_PROCESS_EVERY_N`, `COGNEN_RTSP_TRANSPORT`, `COGNEN_CAMERA_BUFFER`, `API_HOST`, `API_PORT`

---

17. Testing & Evaluation

```
# All 61 tests
python3 -m pytest tests/ -v

# Evaluation against ARCHITECTURE.md targets
python3 v2.0/evaluation/evaluate.py \
    --model v2.0/models/behavior_classifier_v2.onnx \
    --seat_map v2.0/data/seat_maps/room_default.json
```

---

18. Performance Metrics

Trained on 42,600 synthetic windows (AR(1) temporal autocorrelation). Tested on 3,000 independent windows.

Metric	Score	Target	✓
Behavior accuracy	**1.000**	≥ 0.85	✓
F1-macro	**1.000**	≥ 0.80	✓
Precision (using_phone)	**1.000**	≥ 0.90	✓

Recall (attentive_reading)	**1.000**	≥ 0.85	✓
Temporal consistency	**1.000**	≥ 0.90	✓
Tracking MOTA	**1.000**	≥ 0.75	✓
Tracking IDF1	**1.000**	≥ 0.80	✓
Seat occupancy accuracy	**0.963**	≥ 0.95	✓
Inference latency P95	**< 0.1 ms**	< 100 ms	✓

> *Real-classroom accuracy estimated at 0.82–0.88 pending DAiSEE/EngageNet fine-tuning.*

System Performance Targets

Metric	Target
Latency	< 100ms/frame (GPU)
FPS	15 FPS (server)
GPU memory	< 4 GB

---

19. Datasets

Dataset	Purpose	Size
DAiSEE	Engagement detection	9K videos, 4 levels
EngageNet	Student engagement	127K frames
COCO	Person + pose pretraining	330K images
Open Images v7	Object detection	9M images
MPII Human Pose	Pose estimation	25K images
MOT20	Multi-object tracking	8 sequences
Custom classroom	Fine-tuning	Collect 50–100 hours

---

20. Privacy

1. **No raw video stored** — only derived feature vectors and scores
2. **On-premise** — all inference local; no cloud API calls
3. **Anonymization** — track IDs by default; name mapping opt-in
4. **Retention** — auto-purge after `DATA_RETENTION_HOURS` (default 24h)

---

21. Cross-Reference: v1.0

v2.0 reuses v1.0's per-person analysis pipeline. See v1.0 README for browser deployment details.

Component	v1.0 (Browser)	v2.0 (Server)
Face detection	face-api.js	YOLOv8-m + MediaPipe
Landmarks	face-api.js 68-pt	MediaPipe → ibug 68-pt

EAR/MAR/Gaze	TypeScript	Python port
Object detection	YOLOv8-nano ONNX.js	YOLOv8-s Ultralytics
Temporal classifier	Browser ONNX	Server ONNX
Tracking	Single person	ByteTrack multi-person
Seat mapping	N/A	Shapely polygons
Posture analysis	N/A	YOLOv8-pose keypoints
Deployment	Browser + Chrome Ext	FastAPI + GPU